File types

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

This file was found among some files marked confidential but my pdf reader cannot read it, maybe yours can.

💡 Hints

- Remember that some file types can contain and nest other files

---

🧰 Tools Used

- `file` – Determine file type
- `mv` – Rename files
- `chmod` – Change file permissions
- `sh` – Execute shell archive scripts
- `sharutils` – Handle shell archives (shar, uudecode)
- `ar` – Extract ar archives
- `cpio` – Extract cpio archives
- `bzip2` – Decompress bzip2 files
- `gzip` – Decompress gzip files
- `lzip` – Decompress lzip files
- `lz4` – Decompress LZ4 files
- `lzma` – Decompress LZMA files
- `lzop` – Decompress lzop files
- `xz` – Decompress XZ files
- `xxd` – Convert hexadecimal to text

🔍 Complete Solution

Step 1: Download and Prepare

Download the file named `Flag.pdf` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file Flag.pdf
```

Expected output:

```
Flag.pdf: shell archive text
```

The file is not a PDF at all – it's a **shell archive** (`.shar` file).

Step 3: Convert and Execute the Shell Archive

Rename the file to a proper shell script extension:

```
mv Flag.pdf Flag.sh
```

Make it executable:

```
chmod +x Flag.sh
```

View the first few lines to understand what it does:

```
cat Flag.sh | head
```

Output:

```
#!/bin/sh
# This is a shell archive (produced by GNU sharutils 4.15.2).
# To extract the files from this archive, save it to some FILE, remove
# everything before the '#!/bin/sh' line above, then type 'sh FILE'.
#
lock_dir=_sh00046
# Made on 2023-03-16 01:40 UTC by <root@e023b7dee37c>.
# Source directory was '/app'.
#
# Existing files will *not* be overwritten, unless '-c' is specified.
```

Now run the script:

```
./Flag.sh
```

Output:

```
x - created lock directory _sh00046.
x - extracting flag (text)
./Flag.pdf: 119: uudecode: not found
restore of flag failed
flag: MD5 check failed
x - removed lock directory _sh00046.
```

The script fails because `uudecode` is missing. This is part of the `sharutils` package.

Step 4: Install sharutils

```
sudo apt-get install sharutils
```

Now run the script again:

```
./Flag.sh
```

Output:

```
x - created lock directory _sh00046.
x - extracting flag (text)
x - removed lock directory _sh00046.
```

Success! A file named `flag` was extracted.

Step 5: Identify the Extracted File

```
file flag
```

Output:

```
flag: current ar archive
```

The file is an **ar archive** (a Unix static library format).

Step 6: Extract the ar Archive

```
ar x flag
```

This extracts the contents of the ar archive. Let's see what we got:

```
file flag
```

Output:

```
flag: cpio archive
```

Now it's a **cpio archive**!

Step 7: Extract the cpio Archive

```
cpio -i < flag
```

Output:

```
cpio: flag not created: newer or same age version exists
```

```
2 blocks
```

The error occurs because there's already a file named `flag`. Let's rename it and try again:

```
mv flag flag.cpio
cpio -i < flag.cpio
```

Output:

```
2 blocks
```

Now check the new file:

```
file flag
```

Output:

```
flag: bzip2 compressed data, block size = 900k
```

Step 8: Decompress bzip2

```
bzip2 -d flag
```

Output:

```
bzip2: Can't guess original name for flag -- using flag.out
```

Decompression creates `flag.out`.

```
file flag.out
```

Output:

```
flag.out: gzip compressed data, was "flag", last modified: Tue Mar 15 06:50:39 2022,
from Unix, original size modulo 2^32 328
```

Step 9: Decompress gzip

```
gzip -d flag.out
```

Output:

```
gzip: flag.out: unknown suffix -- ignored
```

Gzip expects a `.gz` extension. Rename and try again:

```
mv flag.out flag.gz
gzip -d flag.gz
```

Output:

```
gzip: flag: Value too large for defined data type
```

Despite the warning, a new `flag` file is created.

```
file flag
```

Output:

```
flag: lzip compressed data, version: 1
```

Step 10: Decompress lzip

First, install lzip if needed:

```
sudo apt-get install lzip
```

```
lzip -d flag
```

This creates `flag.out` .

```
file flag.out
```

Output:

```
flag.out: LZ4 compressed data (v1.4+)
```

Step 11: Decompress LZ4

Install lz4:

```
sudo apt install lz4
```

```
lz4 -d flag.out newflag
```

Output:

```
flag.out             : decoded 266 bytes
```

Check the new file:

```
file newflag
```

Output:

```
newflag: LZMA compressed data, non-streamed, size 254
```

Step 12: Decompress LZMA

```
lzma -d newflag
```

Output:

```
lzma: newflag: Filename has an unknown suffix, skipping
```

Rename and try again:

```
mv newflag newflag.lzma
lzma -d newflag.lzma
```

Output:

```
lzma: newflag: Cannot set the file permissions: Value too large for defined data type
```

Despite the warning, check the file:

```
file newflag
```

Output:

```
newflag: lzop compressed data - version 1.040, LZO1X-1, os: Unix
```

Step 13: Decompress lzop

Install lzop:

```
sudo apt install lzop
```

```
lzop -d newflag
```

Output:

```
lzop: newflag: unknown suffix -- ignored
skipping newflag [newflag.raw]
```

Rename and try again:

```
mv newflag newflag.lzop
lzop -d newflag.lzop
```

Now check the file:

```
file newflag
```

Output:

```
newflag: lzip compressed data, version: 1
```

Step 14: Decompress lzip Again

```
lzip -d newflag
```

This creates `newflag.out`.

```
file newflag.out
```

Output:

```
newflag.out: XZ compressed data, checksum CRC64
```

Step 15: Decompress XZ

```
xz -d newflag.out
```

Output:

```
xz: newflag.out: Filename has an unknown suffix, skipping
```

Rename and try again:

```
mv newflag.out newflag.xz
xz -d newflag.xz
```

Output:

```
xz: newflag: Cannot set the file permissions: Value too large for defined data type
```

Now check the final file:

```
file newflag
```

Output:

```
newflag: ASCII text
```

Step 16: Read the Final Flag

```
cat newflag
```

Output:

```
7069636f4354467b66316c656e406d335f6d406e3170756c407431306e5f
6630725f306232637572313779 5f3739623031633236 7d0a
```

This is a **hexadecimal string**! Decode it:

```
cat newflag | xxd -r -p
```

Output:

```
picoCTF{<REDACTED>}
```

📌 Summary of the Extraction Chain

Step	File Type	Command
1	Shell archive (shar)	`./Flag.sh`
2	ar archive	`ar x flag`
3	cpio archive	`cpio -i < flag.cpio`
4	bzip2	`bzip2 -d flag`
5	gzip	`gzip -d flag.gz`
6	lzip	`lzip -d flag`
7	LZ4	`lz4 -d flag.out newflag`
8	LZMA	`lzma -d newflag.lzma`
9	lzop	`lzop -d newflag.lzop`

Step	File Type	Command
10	lzip (again)	`lzip -d newflag`
11	XZ	`xz -d newflag.xz`
12	ASCII (hex)	`xxd -r -p`

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, following these steps will reveal the complete flag.

🧠 Key Concepts

Concept	Description
Shell Archive (shar)	A shell script that extracts files (like a self-extracting archive).
ar Archive	A Unix static library format, similar to `.lib` on Windows.
cpio Archive	A Unix archive format for copying files (Copy In/Copy Out).
bzip2	A block-sorting compression algorithm (higher compression than gzip).
gzip	The standard GNU compression tool (`.gz` extension).
lzip	A LZMA-based compressor with a better integrity check.
LZ4	A compression algorithm focused on speed (fast decompression).
LZMA	The algorithm used by 7-Zip (high compression ratio).
lzop	A fast LZO-based compressor (optimized for speed).

Concept	Description
XZ	A high-compression format based on LZMA2.
Hexadecimal Encoding	Representing text as pairs of hex digits (e.g., `70` = 'p').
Nested Archives	Archives inside archives – like Russian nesting dolls.

💡 Pro Tips

1. **Always use `file` first** – It identifies the actual file type, regardless of extension.
2. **Keep renaming files** – Many decompression tools require specific extensions (`.gz` , `.bz2` , `.xz`).
3. **Ignore warnings when possible** – Warnings like "Value too large for defined data type" often don't prevent extraction.
4. **The hint says "nest other files"** – Expect multiple layers of compression and archives.
5. **Install missing tools as you go** – Each new format may require installing additional packages.
6. **Save the extraction chain** – Document each step; it's easy to lose track after 10+ decompressions.
7. **Hex strings starting with `70` are "pico"** – Recognizing this pattern helps identify the final flag quickly.

🔗 Alternative Tools

Format	Alternative Command
ar	`ar -x flag`
cpio	`cpio -idv < flag.cpio`
bzip2	`bunzip2 flag`

Format	Alternative Command
gzip	`gunzip flag.gz`
lzip	`lunzip flag`
LZ4	`lz4 -d flag.out newflag`
LZMA	`unlzma newflag.lzma`
lzop	`lzop -d newflag.lzop`
XZ	`unxz newflag.xz`
Hex decode	`xxd -r -p` or `python3 -c "print(bytes.fromhex(open('newflag').read()).decode())"`

References

- ar - Linux manual page
- bzip2 - Linux manual page
- cpio - Linux manual page
- gzip - Linux manual page
- lzip - Linux manual page
- lzma - Linux manual page
- lzop - Linux manual page
- xz - Linux manual page